INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY

Nasarawa State, Nigeria

OT/ICS SECURITY LAB REPORT

MODBUS GUARD EVALUATION - H2 PRODUCTION FACILITY SIMULATION

ACICP302 - ICS Network Security and Attacks

Student Name:	SUNUSI ABDULLAHI
Student ID / Reg No:	C11/26/ACIS/17362
Course Code:	ACICP302
Course Name:	ICS Network Security and Attacks
Department:	Cybersecurity
Programme:	ACIS
Lab Repository:	github.com/icdfa/ACICP302ICS-Network-Security-and-Attacks
Verified Commit:	2a66f9bcf253eb3fe31c68ad52794cd535421da8
Activities Covered:	Activity 00 - Activity 06
Version:	1.0

Executive Summary

This report documents Activities 00 through 06 of the ACICP302 ICS Network Security and Attacks student lab. A loopback-only lab environment was built on an Ubuntu 64-bit VM (VMware Workstation, NAT networking) to host a simulated Modbus/TCP-controlled hydrogen (H2) production facility - an electrolyzer, compressor and H2 storage tank feeding an export line. The lab provides two paths to the same simulated plant: an unguarded direct path (port 5020) and a path mediated by a rule-based Modbus guard (port 5021) that validates every write against an absolute safe range, a maximum per-write step size, and a rolling cumulative-drift limit before relaying it to the plant. Two dashboards (undefended on port 8080, guarded on port 8081) provide a live view of each path.

Three attack scripts were run against both the direct and guarded paths under a controlled test plan: a coil/register hold attack (attack_hold.py), a false-data injection attack (false_data_attack.py), and a reconnaissance scan (recon_scan.py). Results were consistent and clear-cut: on the direct path, 100% of malicious writes reached the plant and pushed process values outside their safe operating range (77/77 unsafe pressure readings for the hold attack; 6/6/6 unsafe temperature/pressure/flow readings for the false-data attack). On the guarded path, the guard rejected every single malicious write (0/76 and 0/6/0/6/0/6 unsafe readings reached the plant) and logged a specific, auditable reason for each rejection. Reconnaissance succeeded identically on both paths, since the guard validates writes only and does not restrict reads.

The guard is therefore demonstrated to be an effective, measurable control against the integrity attacks tested, but its protection is entirely opt-in: any client that connects directly to port 5020 bypasses it completely, and the guard offers no protection against reconnaissance, denial-of-service, or coordinated multi-register (correlated-value) manipulation. Full findings, evidence and a risk register are presented in the sections below.

Activity 00 - Environment Setup

A dedicated Ubuntu 64-bit virtual machine was provisioned in VMware Workstation, with its network adapter set to NAT (the VM shares the host's IP address rather than being bridged onto the physical network), so that the lab's Modbus services are never reachable from outside the host.

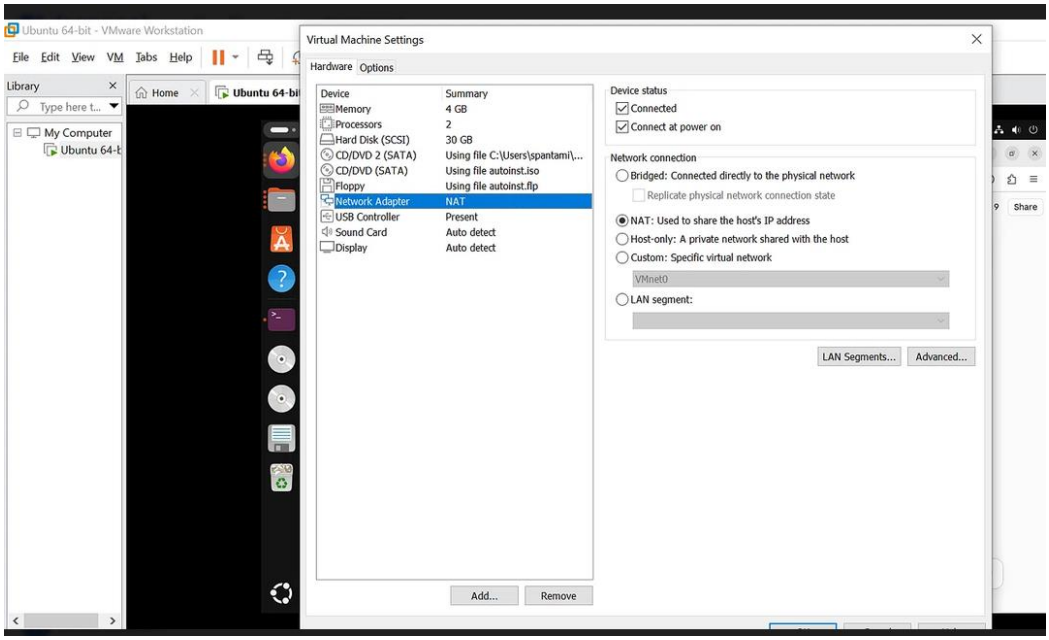


Figure 0.1 - VMware Workstation Virtual Machine Settings: Network Adapter set to NAT, isolating the lab VM from the physical/external network.

The required toolchain was installed via apt, including build-essential, git, python3.14-dev/pip, and the gcc/g++ compiler chain needed to build Python dependencies.

Activity 01 - Source Acquisition and Verification

The official lab repository was cloned into a dedicated course directory:

```
$ mkdir -p ~/course && cd ~/course
$ git clone https://github.com/icdfa/ACICP302ICS-Network-Security-and-Attacks.git
```

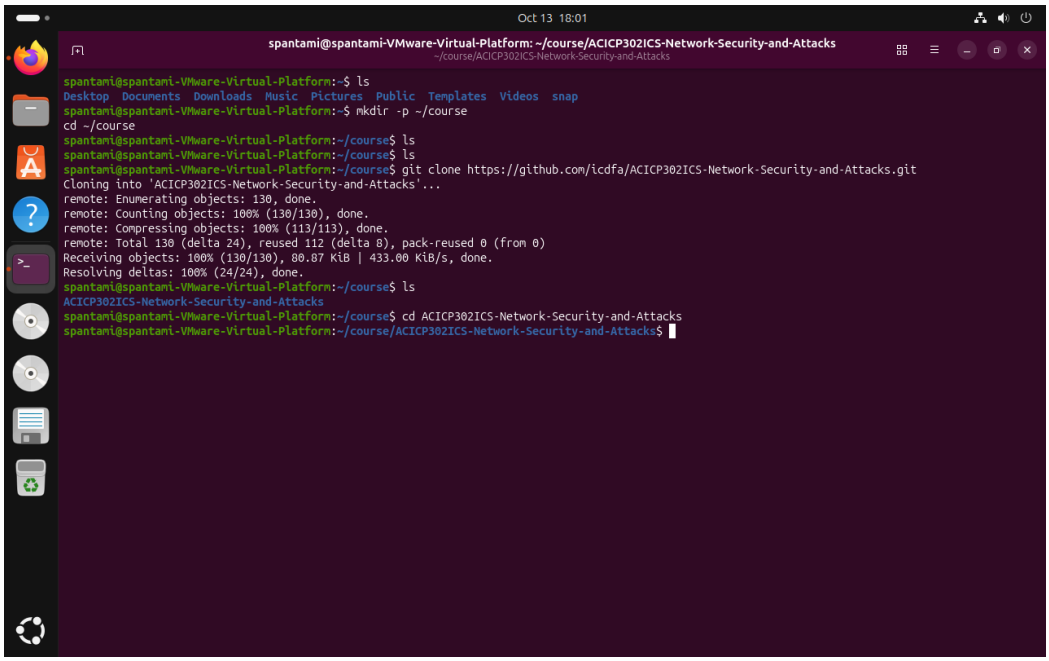


Figure 1.1 - Cloning the official ACICP302ICS-Network-Security-and-Attacks repository (130 objects received, resolved cleanly).

The exact commit in use was captured for integrity/provenance purposes:

```
$ git rev-parse HEAD
2a66f9bcf253eb3fe31c68ad52794cd535421da8
```

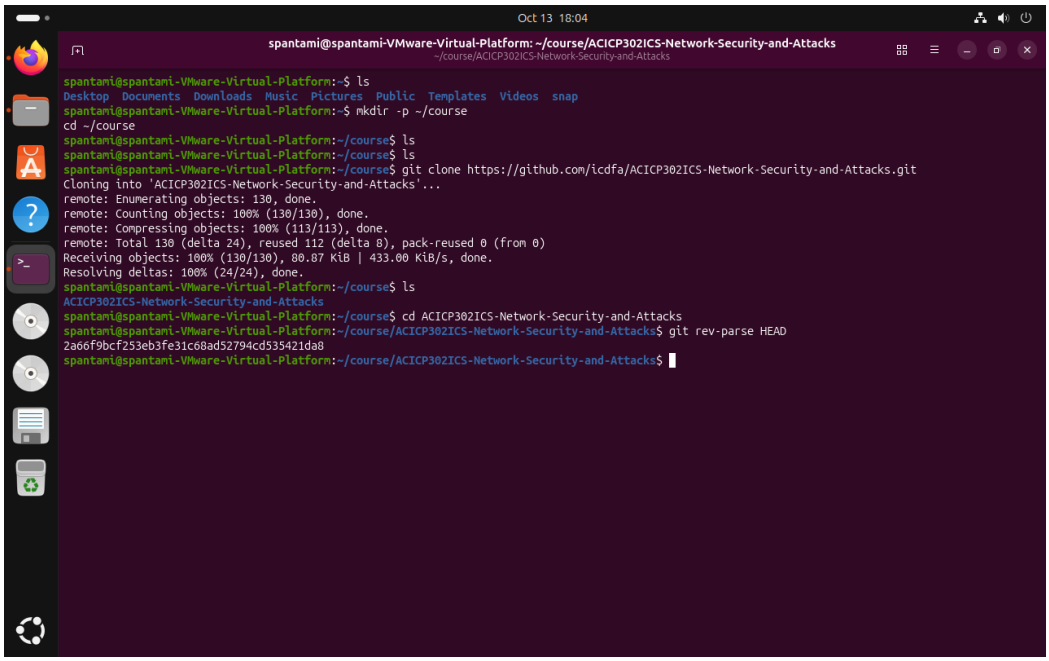


Figure 1.2 - Verified commit hash: 2a66f9bcf253eb3fe31c68ad52794cd535421da8.

A Python virtual environment was created inside the lab's student-lab-source directory and all dependencies were installed from requirements.txt: pymodbus==3.6.9 and Flask>=3.0,<4.0 (with its own dependencies: blinker, click, itsdangerous, jinja2, markupsafe, werkzeug).

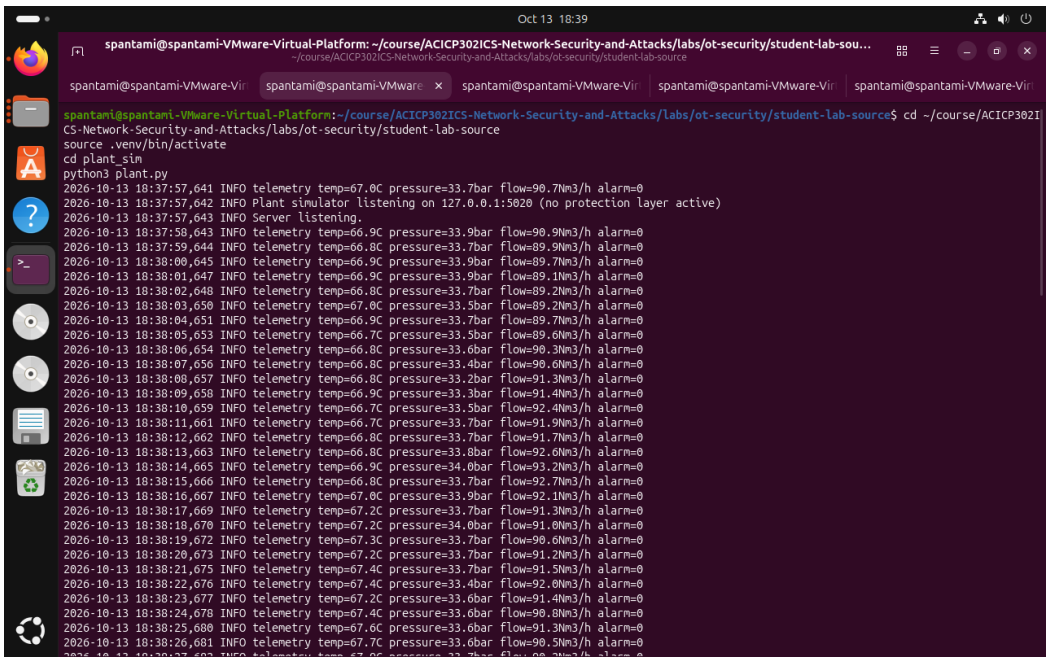


Figure 1.3 - Virtual environment dependency installation: pymodbus 3.6.9 and Flask 3.1.3 installed successfully.

Dedicated evidence folders were created to hold all artefacts generated during testing:

```
$ mkdir -p ~/evidence/{screenshots,logs,pcaps,reports}
```

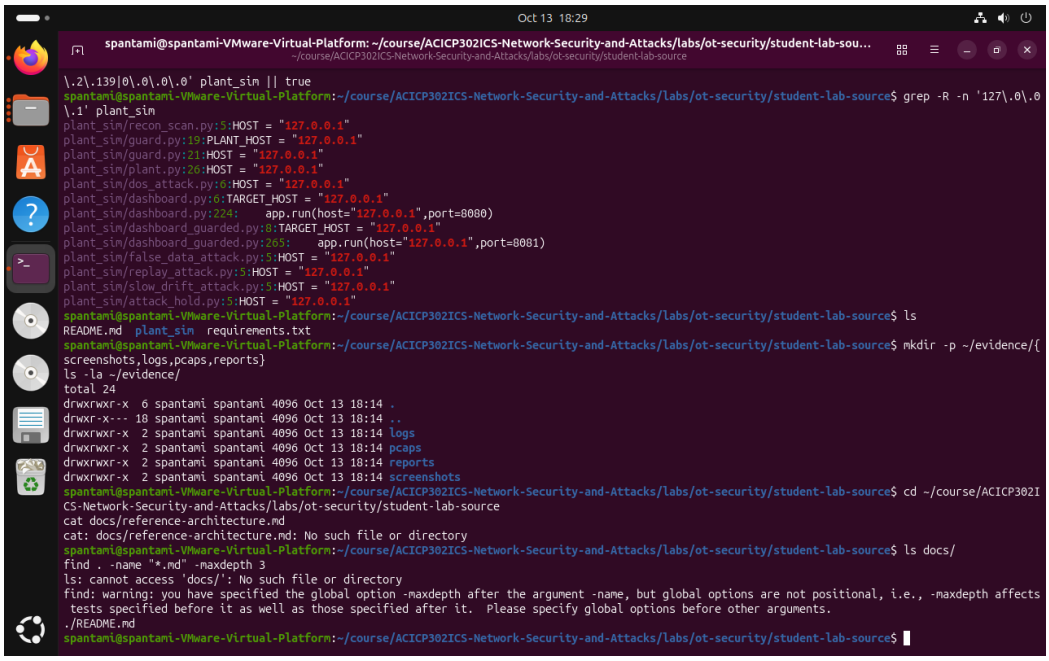


Figure 1.4 - Evidence directory structure created: ~/evidence/{screenshots,logs,pcaps,reports}.

Activity 02 - Architecture and Register Map Discovery

A source-wide search confirmed that every component in the lab - the plant simulator, the guard, both dashboards, and all six attack scripts - binds exclusively to 127.0.0.1 (loopback). No component listens on any externally reachable interface, so the lab is contained entirely within the VM even before the NAT network boundary is considered.

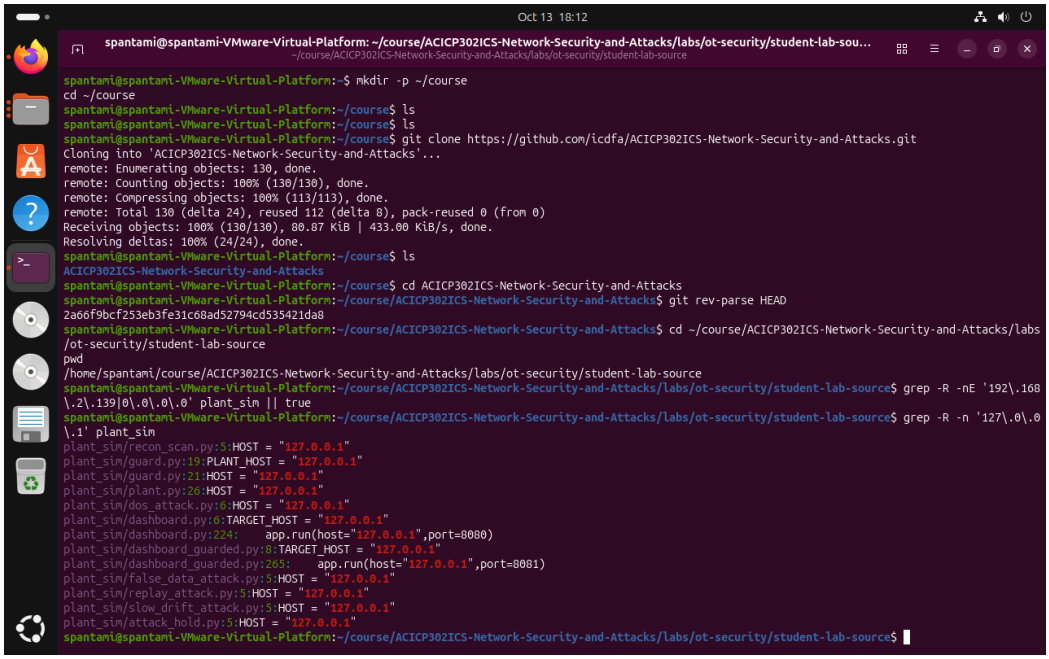


Figure 2.1 - grep confirming every script's HOST/PLANT_HOST is hard-coded to 127.0.0.1 (loopback only).

The lab's component inventory, from the plant_sim/ directory listing:

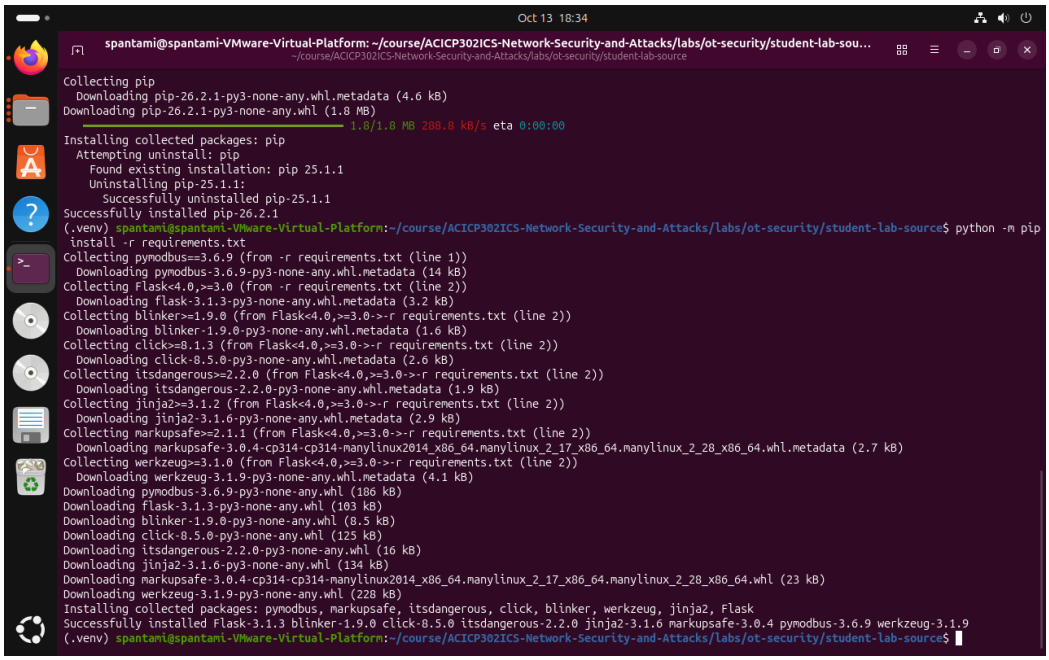


Figure 2.2 - plant_sim/ directory listing: plant.py, guard.py, dashboard.py, dashboard_guarded.py, and the six attack scripts.

- plant.py - the unguarded Modbus/TCP plant simulator, listening on 127.0.0.1:5020.
- guard.py - a rule-based Modbus/TCP guard (proxy), listening on 127.0.0.1:5021, protecting the plant at 127.0.0.1:5020.
- dashboard.py - a Flask UI that reads directly from the plant (port 8080) - the 'undefended' view.
- dashboard_guarded.py - a Flask UI that reads via the guard (port 8081) - the 'protected' view.
- recon_scan.py, false_data_attack.py, slow_drift_attack.py, replay_attack.py, attack_hold.py, dos_attack.py - the six attack scripts. Each defaults to port 5020 but accepts a port argument, so each can be pointed at either the direct path (5020) or the guarded path (5021).

The plant's Modbus holding-register map - ModbusSequentialDataBlock(0, [670, 340, 900, 0] + [0]*96) - was confirmed from source. The guard mirrors this same block inside its own GuardedContext, which is visible in guard.py's main():

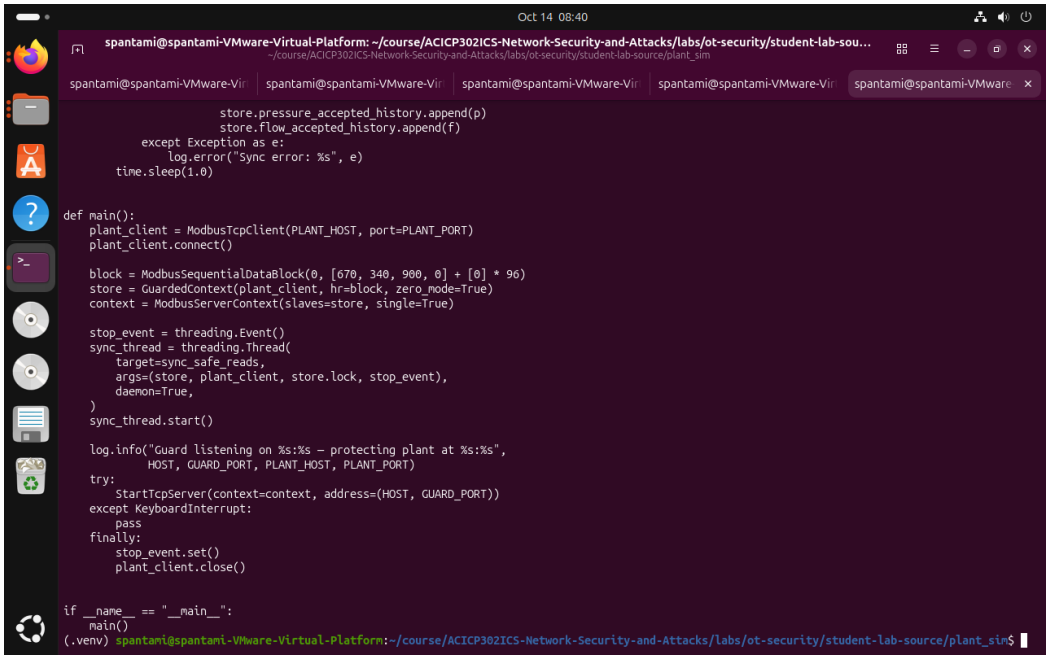


Figure 2.3 - guard.py main(): mirrors the plant's register block [670, 340, 900, 0] + reserved inside a GuardedContext, starts a background sync thread, then serves the validated view on the guard port.

Register	Meaning	Scale	Baseline value
HR0	Electrolyzer Temperature	raw / 10 = deg C	670 -> 67.0 deg C
HR1	H2 Tank Pressure	raw / 10 = bar	340 -> 34.0 bar
HR2	H2 Flow Rate	raw / 10 = Nm3/h	900 -> 90.0 Nm3/h
HR3	Alarm flag	0 = normal, 1 = alarm	0
HR4-HR99	Reserved / unused	-	0

The undefended dashboard reads these same four registers via read_holding_registers(0, 4, slave=1) and maps them directly onto the process-flow diagram and alarm badge shown to the operator:

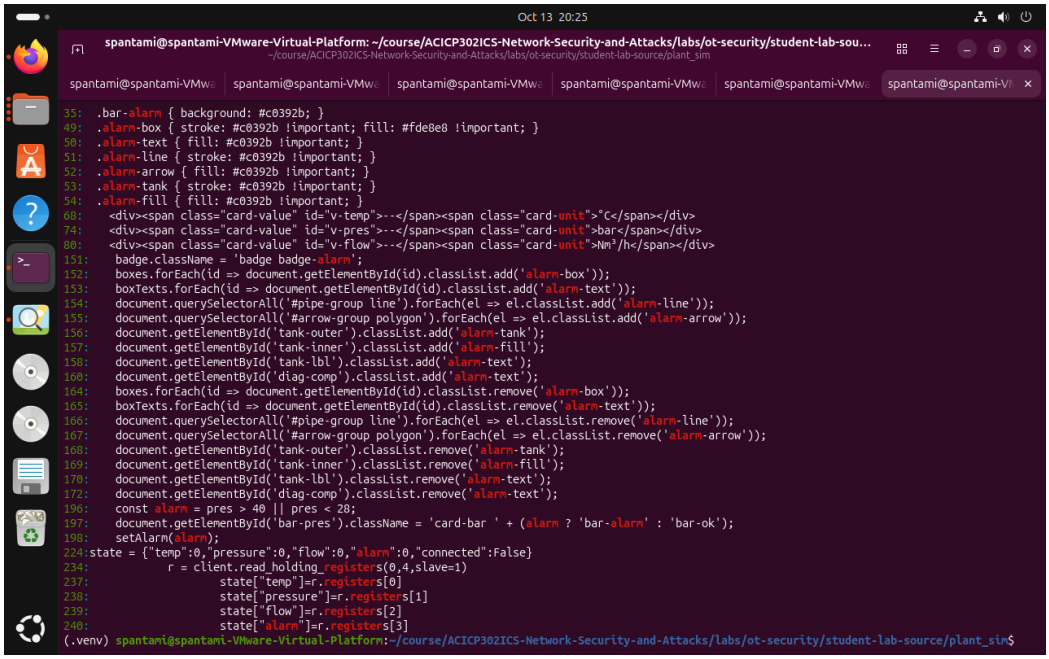


Figure 2.4 - dashboard.py source: alarm CSS classes and the client-side state object mapping read_holding_registers(0,4,slave=1) to temperature/pressure/flow/alarm.

The guard's safe operating thresholds - not visible from the plant's register map alone - were confirmed later from its live rejection events (Activity 05): Temperature 60.0-75.0 deg C, Pressure 30.0-40.0 bar, Flow 50.0-150.0 Nm3/h, with additional per-write step-size limits (3.0 deg C / 5.0 bar / 10.0 Nm3/h) and rolling 3-write cumulative-drift limits (2.0 deg C / 2.0 bar / 4.0 Nm3/h).

Activity 03 - Baseline Operation

All four services were started in separate terminals and confirmed operational before any attack testing began.

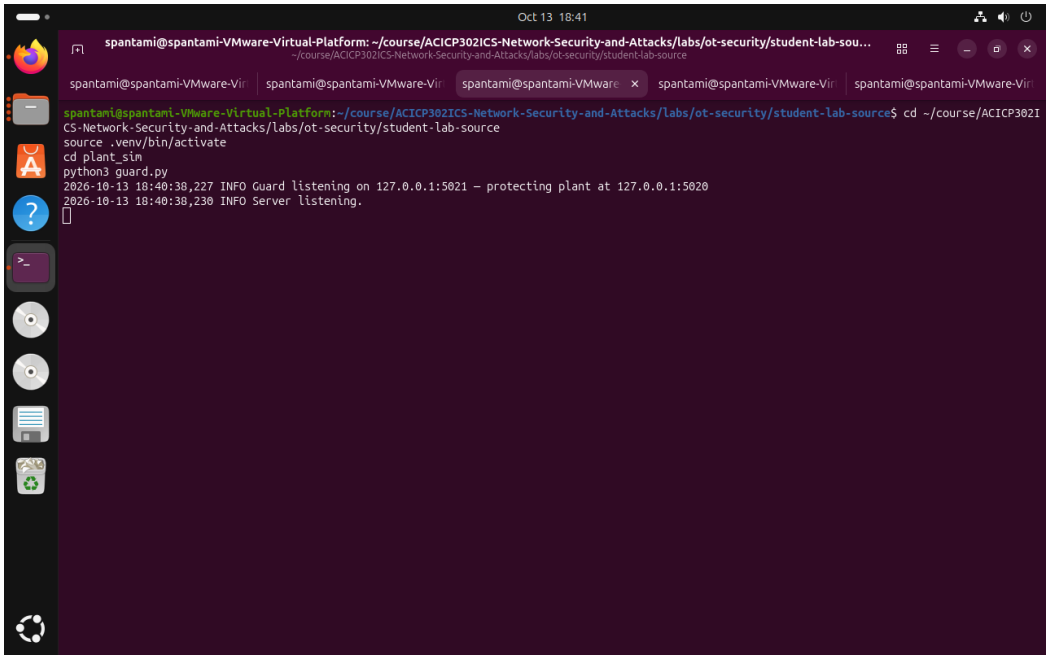


Figure 3.1 - plant.py running: "Plant simulator listening on 127.0.0.1:5020 (no protection layer active)", with live telemetry (temp ~67 deg C, pressure ~34 bar, flow ~90 Nm3/h, alarm=0).

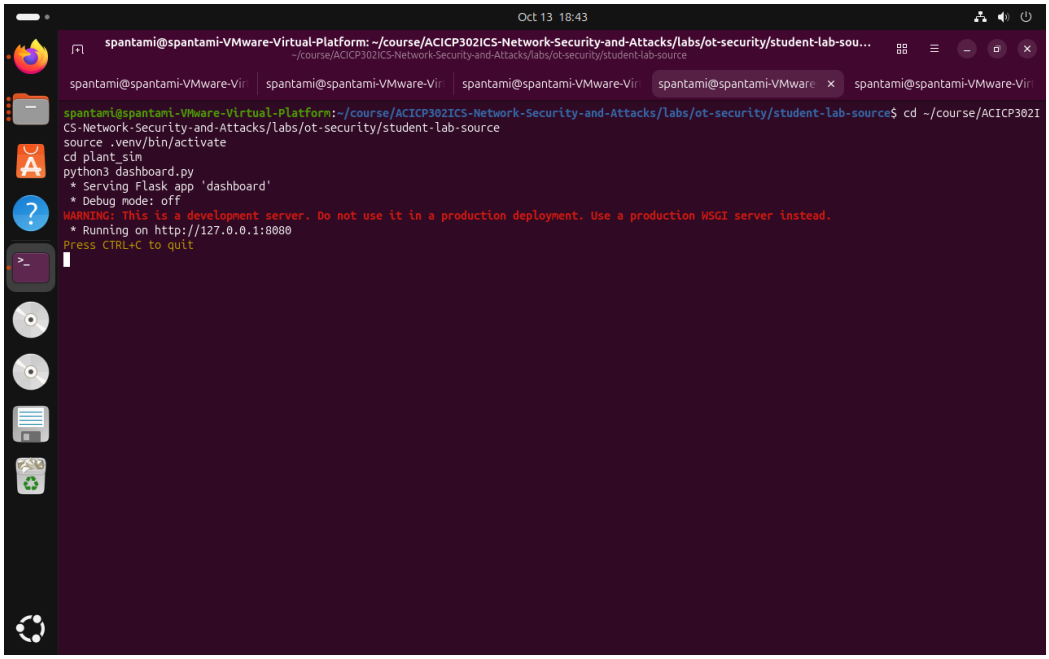


Figure 3.2 - guard.py running: "Guard listening on 127.0.0.1:5021 - protecting plant at 127.0.0.1:5020".

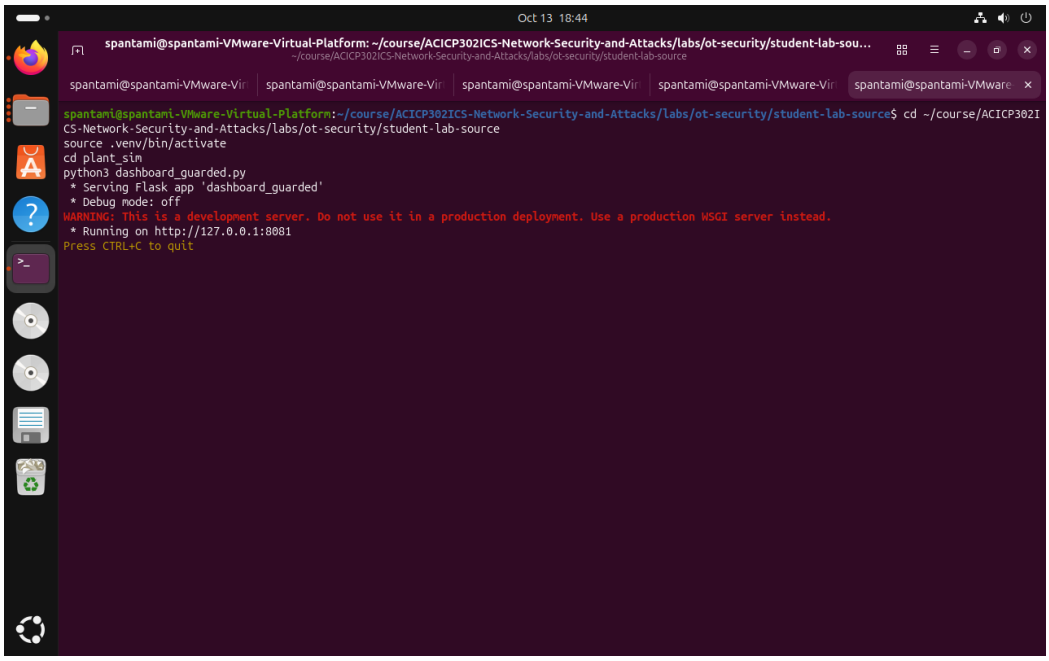


Figure 3.3 - dashboard.py running: Flask development server on 127.0.0.1:8080 (undefended view).

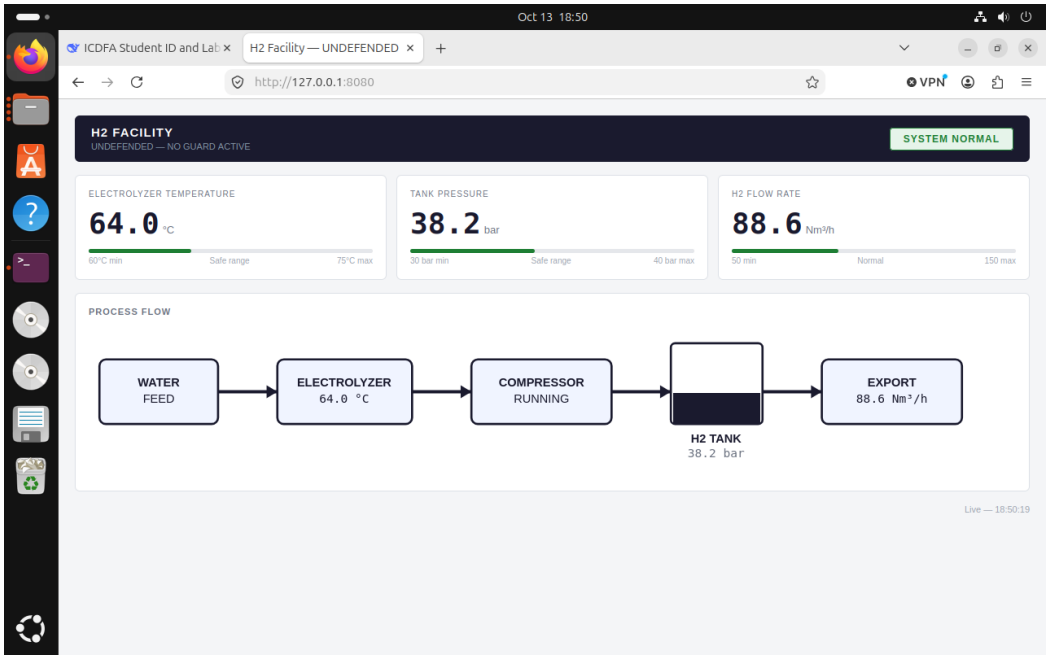


Figure 3.4 - dashboard_guarded.py running: Flask development server on 127.0.0.1:8081 (guarded view).

Listening sockets were independently verified with ss -tulpn, confirming all four services bound only to 127.0.0.1 on their expected ports (5020, 5021, 8080, 8081), each tied to a distinct python3 process.

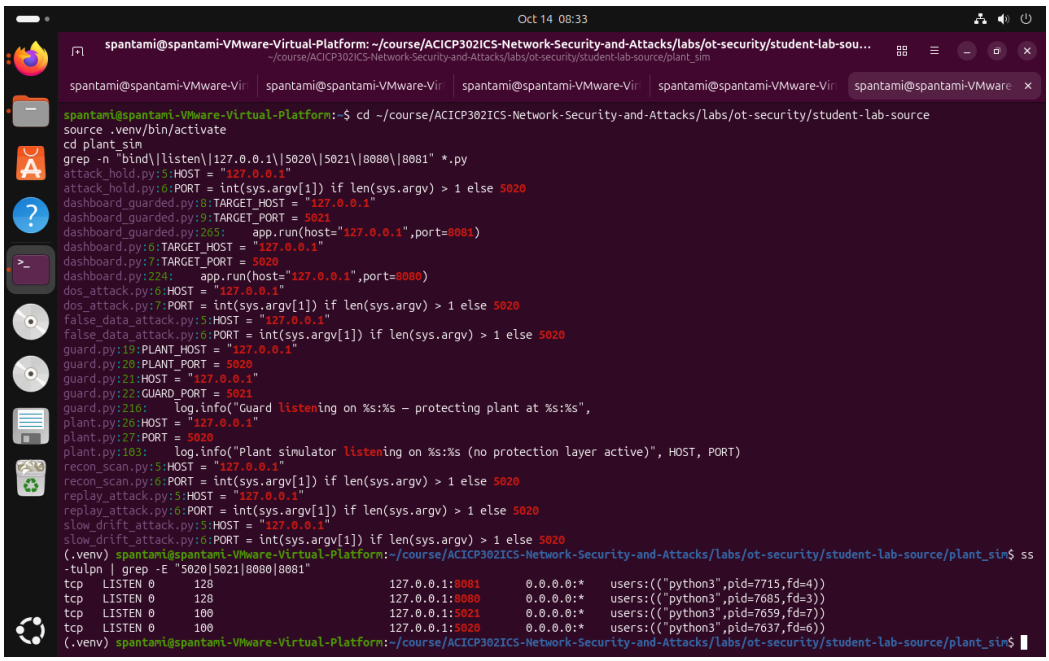


Figure 3.5 - ss -tulpn output confirming 127.0.0.1:5020/5021/8080/8081 all LISTEN, loopback-only.

Baseline dashboard readings were recorded on both views before any attack:

View	Temperature	Pressure	Flow
Undefended (8080) - SYSTEM NORMAL	64.0 deg C	38.2 bar	88.6 Nm3/h
Guarded (8081) - PROTECTED	70.9 deg C	36.1 bar	69.9 Nm3/h

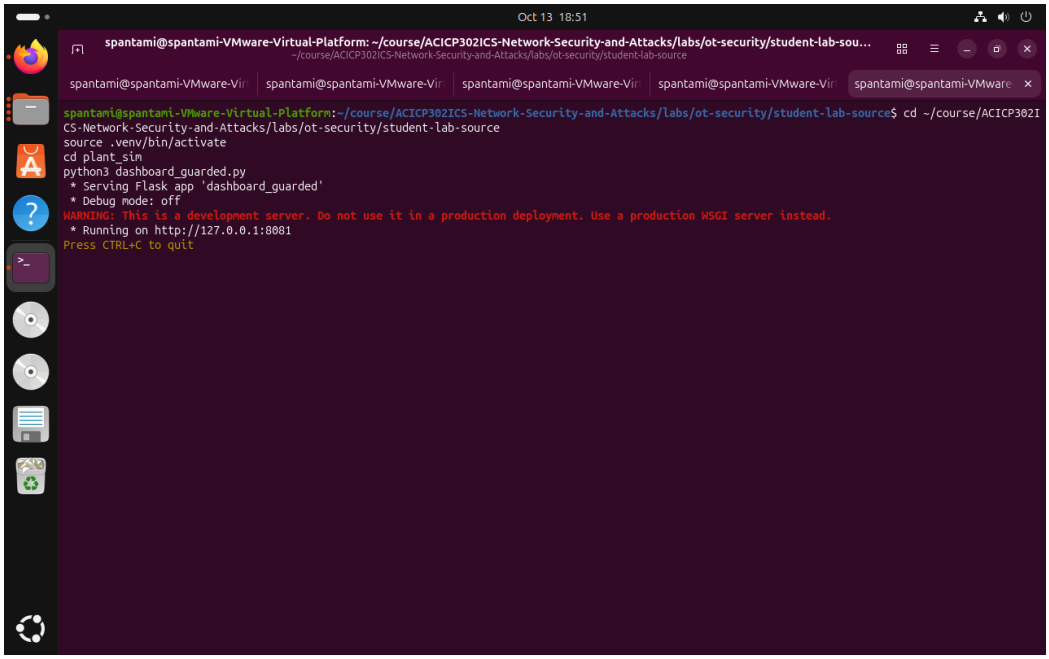


Figure 3.6 - Undefended dashboard (http://127.0.0.1:8080) baseline: SYSTEM NORMAL.

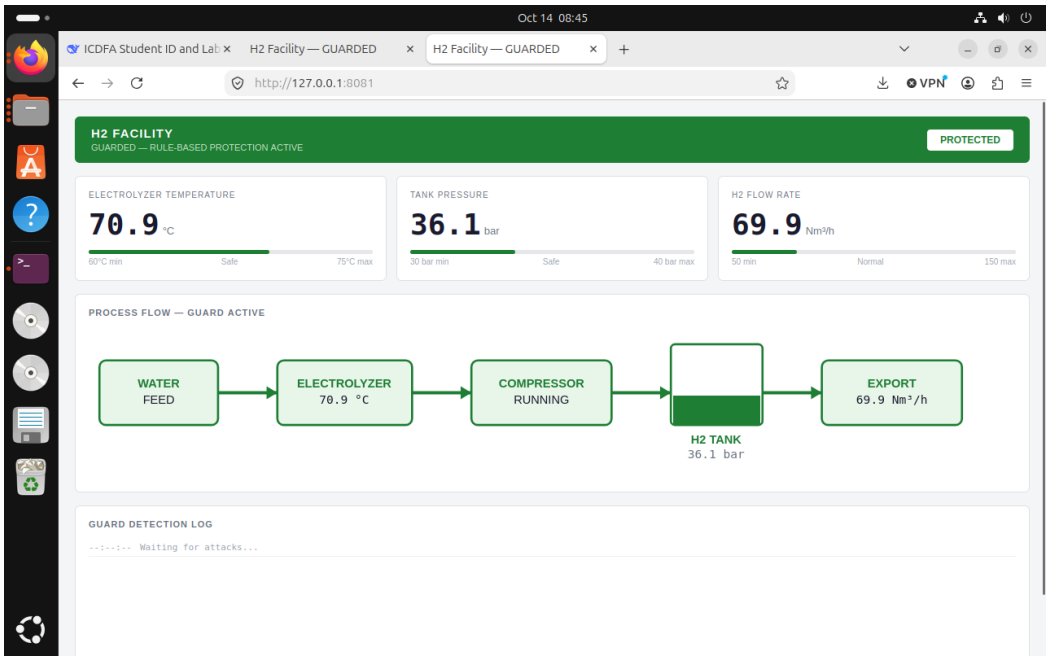


Figure 3.7 - Guarded dashboard (http://127.0.0.1:8081) baseline: PROTECTED, Guard Detection Log reads "Waiting for attacks...".

Baseline Modbus/TCP traffic on the plant's loopback port was captured for evidentiary purposes:

```
$ sudo tcpdump -i lo port 5020 -w ~/evidence/pcaps/modbus_capture.pcap
```

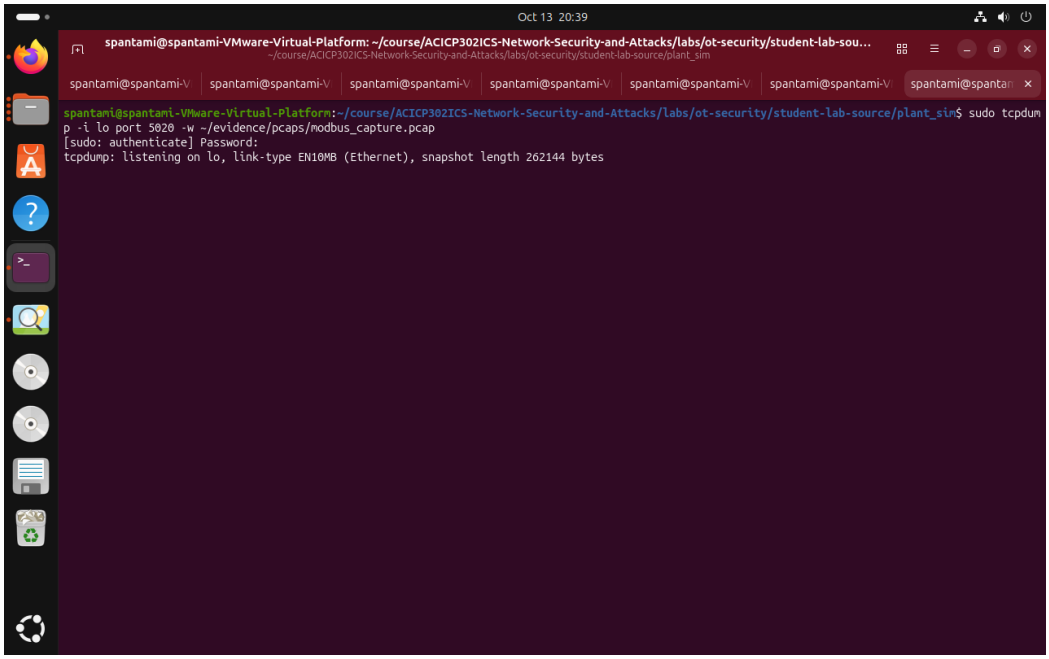


Figure 3.8 - tcpdump capture started on loopback interface, port 5020.

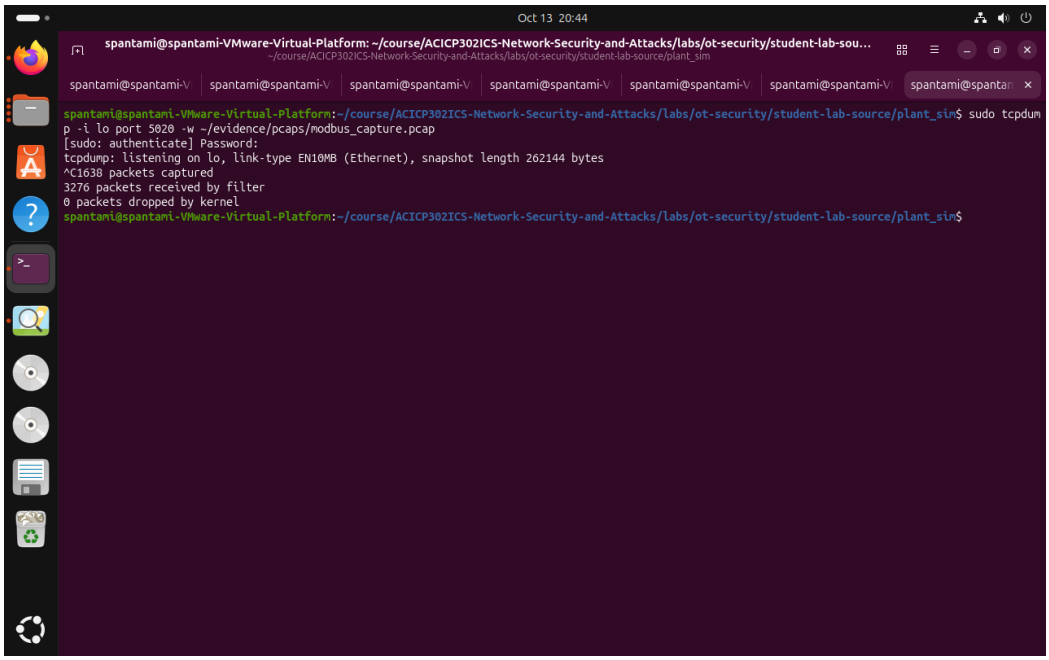


Figure 3.9 - tcpdump capture stopped: 1,638 packets captured, 3,276 received by filter, 0 dropped by kernel; saved to ~/evidence/pcaps/modbus_capture.pcap.

Activity 04 - Risk Register and Guard Analysis

4.1 Service / Risk Register

Component	Port / Protocol	Risk	Control
Plant (direct Modbus)	5020 / Modbus-TCP	Unauthenticated writes possible; process manipulation, safety impact. Priority: High	None - guard not applied on this path
Guard	5021 / Modbus-TCP	Rule bypass possible via direct path; fixed thresholds. Priority: Medium	3 checks: absolute range, step-size, rolling baseline drift

Undefended dashboard	8080 / HTTP	No authentication required; unauthorised process observation. Priority: Medium	Loopback-only bind
Guarded dashboard	8081 / HTTP	No authentication required; unauthorised process observation. Priority: Medium	Loopback-only bind

Recommended controls: add authentication to both dashboards; implement authentication for Modbus writes; add role-based authorisation; monitor guard events in real time; restrict the direct path (5020) to approved clients only.

4.2 Guard Decision Table

Check	Input	Condition	Action / Log
1. Absolute range	Temperature / Pressure / Flow value	Outside 60-75 deg C / 30-40 bar / 50-150 Nm3/h	Reject write; entry written to events.jsonl
2. Step-size	Change between consecutive writes	Step exceeds 3.0 deg C / 5.0 bar / 10.0 Nm3/h	Reject write; entry written to events.jsonl
3. Rolling baseline drift	Cumulative change over last 3 accepted writes	Drift exceeds 2.0 deg C / 2.0 bar / 4.0 Nm3/h	Reject write; entry written to events.jsonl

Notes: the guard is rule-based, not AI-driven; its thresholds are fixed in source rather than configurable at runtime; it applies only to traffic sent through port 5021 (the guarded path) - the direct path (5020) bypasses it entirely (see Figure 2.3).

4.3 Guard Checks to Risk Register Mapping

Guard Check	Addresses Risk	Coverage
1. Absolute range	Out-of-range pressure/temp/flow writes (Plant - Unauthenticated writes)	Partial - guarded path only
2. Step-size	Sudden large changes / process manipulation	Partial - guarded path only
3. Rolling baseline drift	Slow/gradual process manipulation (slow-drift attacks)	Partial - guarded path only

Uncovered risks, not addressed by any guard check: absence of authentication; absence of authorisation; direct-path bypass; availability/denial-of-service; correlated-value (multi-register coordinated) attacks.

4.4 Guard Limitations and Residual Risk

Limitation	Description / Impact	Residual Risk
No authentication	Any client can connect to the guard without credentials; an attacker does not need to bypass authentication because none exists.	High
No authorisation	No role-based access control for writes; any connected client can attempt a write.	High
Direct path bypass	Port 5020 bypasses the guard entirely; an attacker can write straight to the plant with no validation.	Critical
Availability	No protection against denial-of-service or resource exhaustion; the guard may become unavailable under load.	Medium

Correlated values	The guard checks individual register values, not correlations between them; a coordinated multi-register attack may still pass.	Medium
Fixed thresholds	Thresholds are static and may miss attacks engineered to stay just under them (e.g. an even slower drift).	Medium
Not a certified safety system	The guard is educational tooling, not certified for process safety.	Informational

Recommendations: add authentication to the guard and plant; add role-based authorisation; disable or restrict the direct path (5020); add rate limiting for availability protection; implement correlated-value checks; use adaptive/dynamic thresholds; never rely on the guard as a certified safety system.

4.5 Activity 04 Conclusion

- All services bind to 127.0.0.1 (loopback only) - SAFE.
- No external network exposure - SAFE.
- No authentication on any service - RISK.
- Direct path (5020) bypasses the guard - RISK.
- Dashboards have no login - RISK.
- The guard implements 3 checks - PARTIAL CONTROL.

The simulation is safely configured for loopback-only operation. However, within the simulation itself there is no authentication, no authorisation, and the direct path bypasses the guard entirely. These are educational limitations rather than production-ready controls, and are the basis for the attack testing carried out in Activity 05.

Activity 05 - Controlled Attack Testing

5.1 Test Plan

Each attack script was run twice against the loopback plant: once against port 5020 (direct, unguarded) and once against port 5021 (guarded). Dashboard state was recorded before and after each run, and the guard's events.jsonl log was captured as corroborating evidence. Only one scenario was run at a time; services were left running between runs rather than restarted, since the plant's background telemetry loop naturally returns process values toward their normal operating band.

Scenario	Expected (Direct / 5020)	Expected (Guarded / 5021)
1. attack_hold.py	Write accepted	Write rejected, event logged
2. false_data_attack.py	Writes accepted	Writes rejected, events logged
3. recon_scan.py	Reads succeed	Reads succeed (guard does not restrict reads)

5.2 Scenario 1 - attack_hold.py

Direct path (port 5020):

```
--- ATTACK HOLD: targeting port 5020 ---
Writes sent: 77
Peak pressure observed: 95.0 bar at t=0.01s (safe range: 30.0-40.0 bar)
Readings outside safe range: 77/77 (100%)
RESULT: UNSAFE VALUE REACHED THE PLANT
```

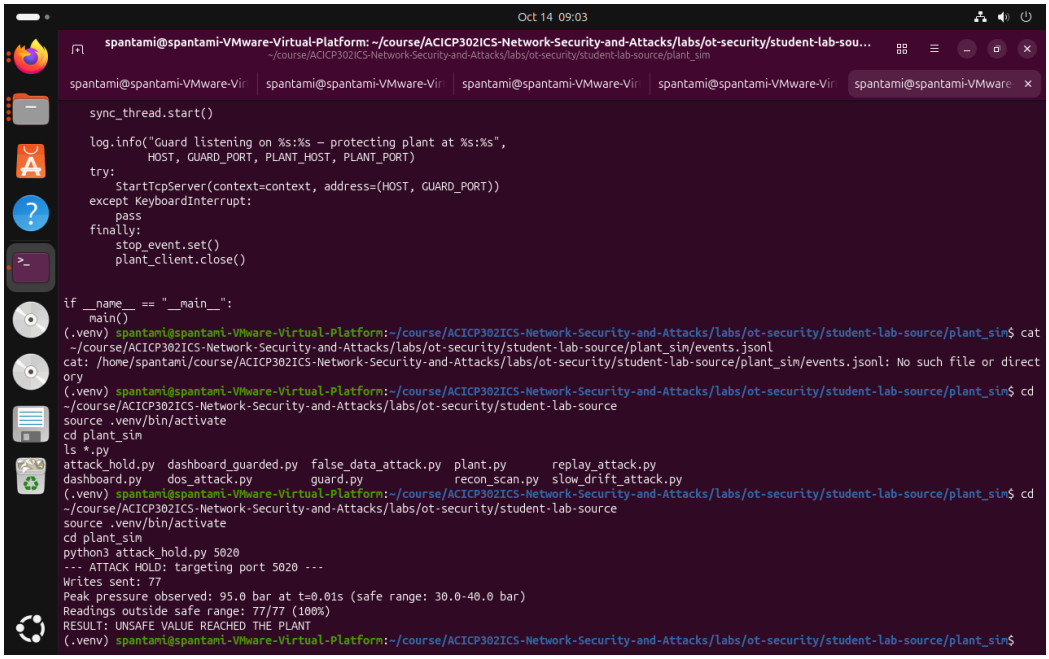


Figure 5.1 - attack_hold.py vs port 5020 (direct): every write accepted, pressure forced to 95.0 bar against a 30-40 bar safe range.

Guarded path (port 5021):

```
--- ATTACK HOLD: targeting port 5021 ---
Writes sent: 76
Peak pressure observed: 39.3 bar at t=0.03s (safe range: 30.0-40.0 bar)
Readings outside safe range: 0/76 (0%)
RESULT: value never left safe range - attack fully contained
```

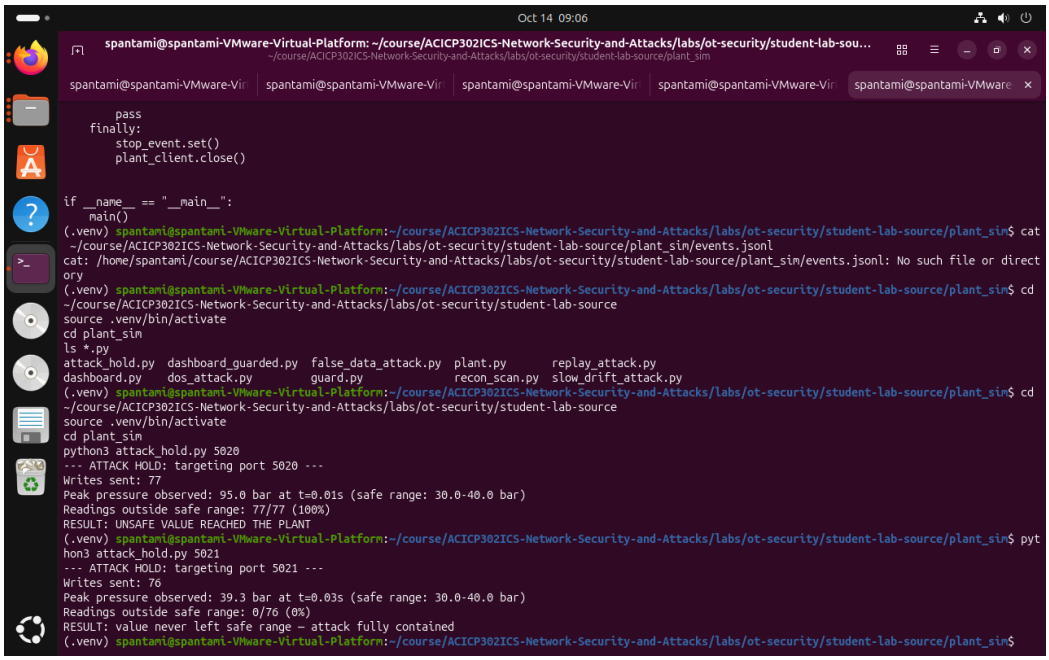


Figure 5.2 - attack_hold.py vs port 5021 (guarded): 0 of 76 writes reached an unsafe value - attack fully contained.

Corresponding guard log entries (events.jsonl), repeated for each rejected write:

```
{"timestamp": "2026-10-14 09:04:42", "reason": "95.0 bar outside safe range 30.0-40.0 bar; step of 56.4 bar exceeds max 5.0 bar; cumulative drift of 56.2 bar over last 3 accepted writes exceeds 2.0 bar limit"}
```

Conclusion: on the direct path, 100% of the hold attack's writes reached the plant and no event was logged (the path bypasses the guard entirely). On the guarded path, 0% of writes reached the plant; every rejection was logged with the specific rule(s) it tripped (absolute range, step-size, and cumulative drift all fired together).

5.3 Scenario 2 - false_data_attack.py

Direct path (port 5020):

```
--- FALSE DATA INJECTION: port 5020 ---
Peak temp observed:     110.0C at t=0.00s (safe: 60.0-75.0C)
Peak pressure observed: 55.0bar at t=0.00s (safe: 30.0-40.0bar)
Peak flow observed:     200.0Nm3/h at t=0.00s (safe: 50.0-150.0)
Unsafe readings: temp 6/6, pressure 6/6, flow 6/6
RESULT: UNSAFE VALUES REACHED THE PLANT
```

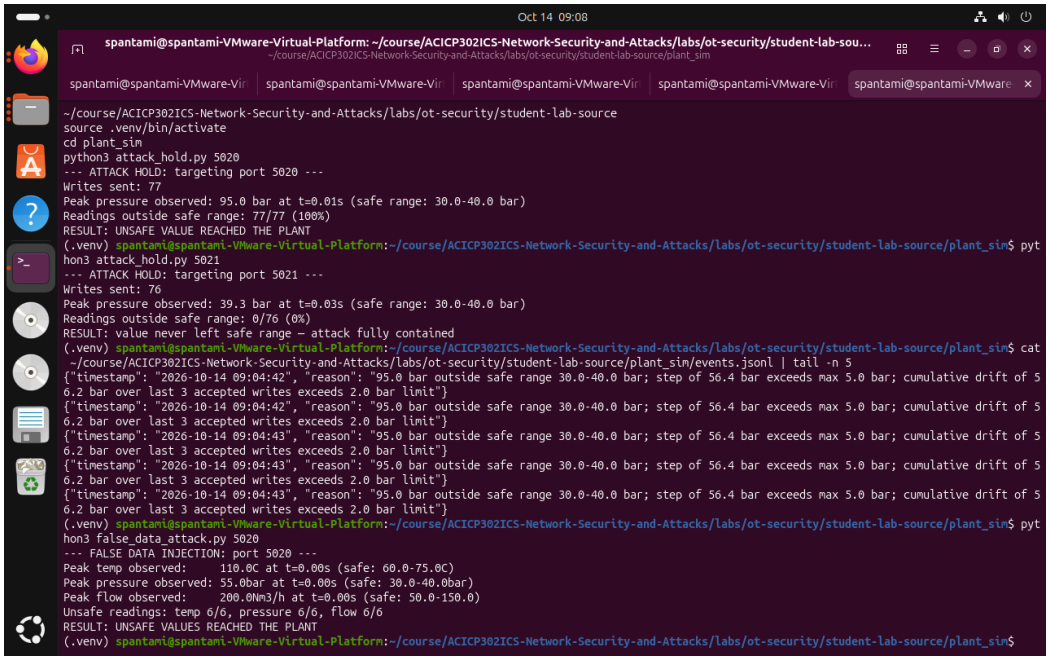


Figure 5.3 - false_data_attack.py vs port 5020 (direct): all 6/6 readings across temperature, pressure and flow pushed outside their safe ranges.

Guarded path (port 5021):

```
--- FALSE DATA INJECTION: port 5021 ---
Peak temp observed:     74.4C at t=1.02s (safe: 60.0-75.0C)
Peak pressure observed: 38.2bar at t=2.04s (safe: 30.0-40.0bar)
Peak flow observed:     149.0Nm3/h at t=3.05s (safe: 50.0-150.0)
Unsafe readings: temp 0/6, pressure 0/6, flow 0/6
RESULT: attack fully contained
```

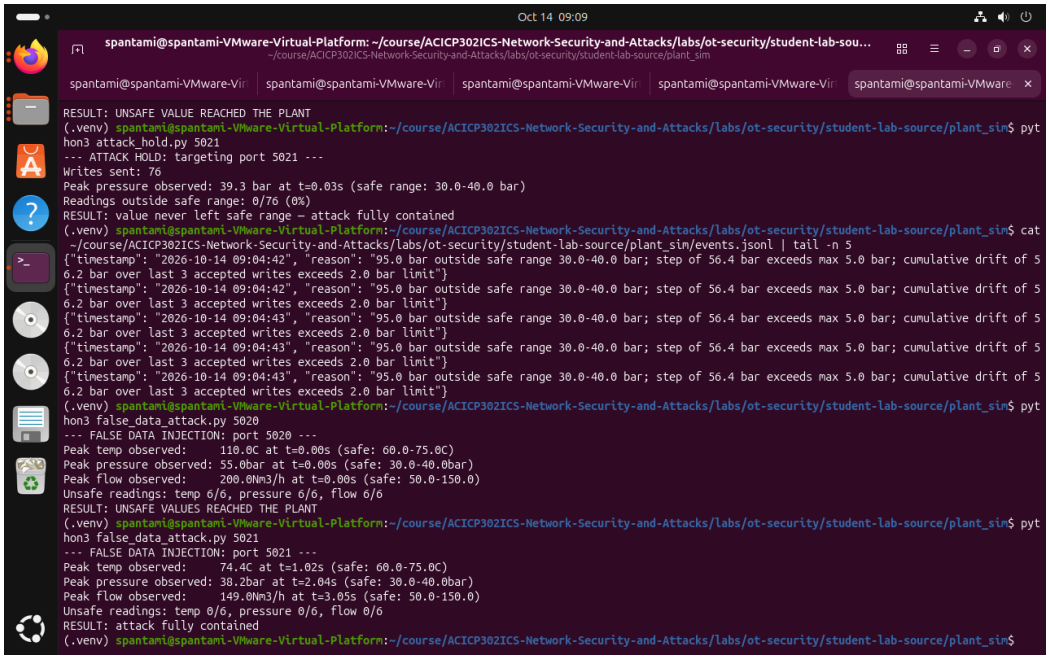


Figure 5.4 - false_data_attack.py vs port 5021 (guarded): 0/6/0/6/0/6 unsafe readings - attack fully contained.

Corresponding guard log entries show each rejected write with its specific reason, e.g.:

```
{"timestamp": "2026-10-14 09:09:22", "reason": "110.0C outside safe range 60.0-75.0C;
step of 35.8C exceeds max 3.0C; cumulative drift of 35.6C over last 3 accepted writes
exceeds 2.0C limit"}
{"timestamp": "2026-10-14 09:09:22", "reason": "55.0 bar outside safe range 30.0-40.0
bar; step of 16.8 bar exceeds max 5.0 bar; cumulative drift of 17.1 bar over last 3
accepted writes exceeds 2.0 bar limit"}
{"timestamp": "2026-10-14 09:09:22", "reason": "200.0 Nm3/h outside safe range 50.0-150.0
Nm3/h; step of 51.4 Nm3/h exceeds max 10.0 Nm3/h; cumulative drift of 51.8 Nm3/h over
last 3 accepted writes exceeds 4.0 Nm3/h limit"}
```

Conclusion: identical pattern to Scenario 1 - complete compromise of all three process variables on the direct path, complete containment on the guarded path, with every rejection individually logged and attributable to a specific rule.

5.4 Scenario 3 - recon_scan.py

Direct path (port 5020):

```
--- RECON SCAN: port 5020 ---
  Regs 0-9: [720, 389, 1419, 0, 0, 0, 0, 0, 0, 0]
  Regs 10-49: all 0
```

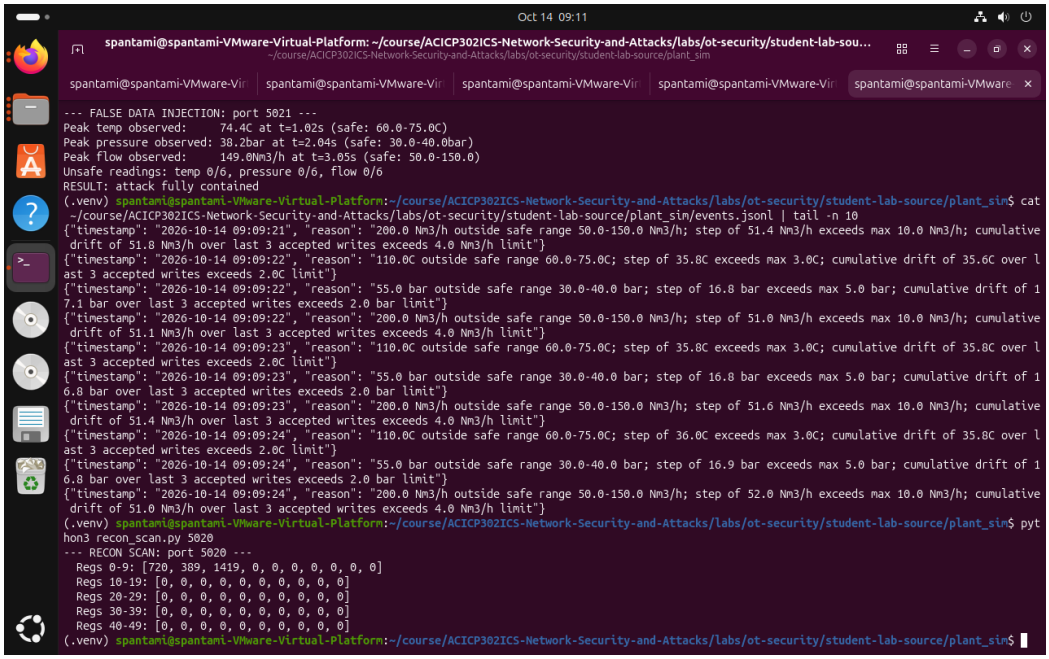


Figure 5.5 - recon_scan.py vs port 5020 (direct): register read succeeds, returning 72.0 deg C / 38.9 bar / 141.9 Nm3/h / alarm 0 to an unauthenticated client.

Guarded path (port 5021):

```
--- RECON SCAN: port 5021 ---
  Regs 0-9: [702, 368, 1399, 0, 0, 0, 0, 0, 0, 0]
  Regs 10-49: all 0
```

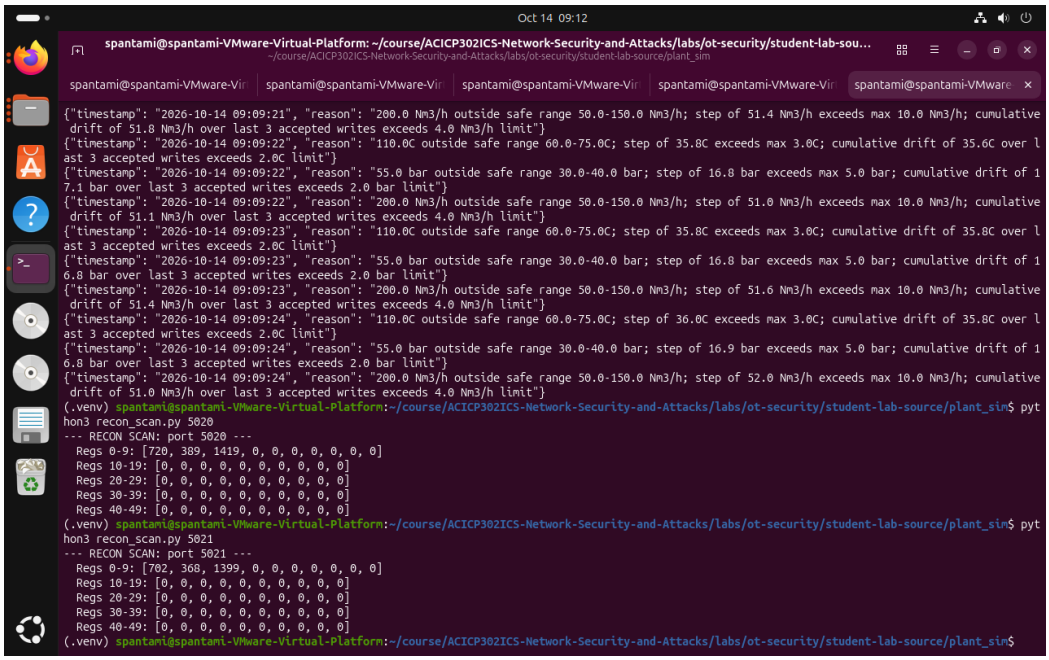


Figure 5.6 - recon_scan.py vs port 5021 (guarded): register read also succeeds - near-identical process state disclosed.

Conclusion: the guard validates writes only. It does not restrict, rate-limit, or even log read requests, so reconnaissance - full disclosure of the plant's live process state to any unauthenticated client - succeeds equally well through either path.

5.5 Expected vs. Observed Comparison

Scenario	Path	Observed	Match?

1. attack_hold.py	Direct (5020)	All 77 writes accepted; peak 95.0 bar (unsafe) reached the plant	Yes
	Guarded (5021)	0 of 76 writes reached an unsafe value; every rejection logged with reason	Yes
2. false_data_attack.py	Direct (5020)	6/6 readings unsafe across temp/pressure/flow; all reached the plant	Yes
	Guarded (5021)	0/6 unsafe readings on all three channels; every rejection logged	Yes
3. recon_scan.py	Direct (5020)	Register reads succeeded; full process state disclosed	Yes
	Guarded (5021)	Register reads also succeeded; guard does not restrict reads	Yes

Overall conclusion for Activity 05: the guard successfully rejected 100% of the malicious writes tested on the guarded path, while the direct path allowed 100% of them through without any validation. Reconnaissance succeeded identically on both paths, confirming the guard's protection is limited to write-integrity and does not extend to confidentiality.

Activity 06 - Recovery Verification

After the attack scripts completed, the undefended dashboard was rechecked and had returned to a normal reading - Temperature 67.1 deg C, Pressure 39.5 bar, Flow 76.3 Nm3/h, status "SYSTEM NORMAL" - within the simulator's natural operating band.

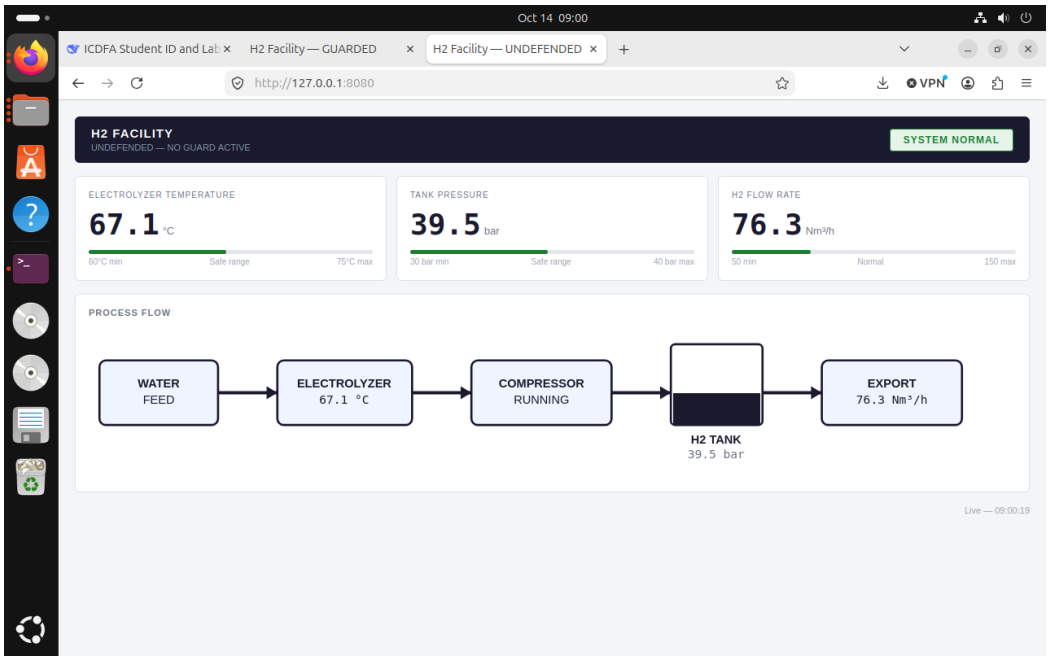


Figure 6.1 - Undefended dashboard post-attack: SYSTEM NORMAL, readings back within the plant's normal operating band.

Item	Result
Dashboard status	Normal / baseline (both undefended and guarded views)
Services running	Yes - plant.py, guard.py, dashboard.py, dashboard_guarded.py all still active
Ports listening	Yes - 127.0.0.1:5020/5021/8080/8081 re-confirmed via ss -tulpn
Recovery method	No manual restart was required. On the guarded path, no malicious write was ever accepted, so the plant state was never disturbed. On the

	direct path, the plant's own background telemetry loop overwrote the injected values on its next normal update cycle, returning readings to the simulator's natural operating band.
Recovery verified	Yes

Evidence Index

Full raw evidence - screenshots, terminal logs, the Modbus packet capture (modbus_capture.pcap) and the guard's events.jsonl log - is retained in ~/evidence/{screenshots,logs,pcaps,reports} on the lab VM. The table below indexes every figure included in this report.

Fig.	Description	Activity
0.1	VMware NAT network setting	00
1.1-1.4	Repo clone, commit hash, dependency install, evidence folders	01
2.1-2.4	Register map grep, component listing, guard.py register/sync logic, dashboard.py source	02
3.1-3.9	All 4 services running, ss -tulpn, baseline dashboards, tcpdump capture	03
5.1-5.6	attack_hold.py, false_data_attack.py and recon_scan.py vs both ports	05
6.1	Post-attack dashboard recovery	06

Overall Conclusion

This lab demonstrated, with reproducible and quantified evidence, that a purpose-built Modbus guard enforcing absolute-range, step-size and rolling-drift checks can completely contain both a coil/register hold (lockout) attack and a false-data-injection attack against a simulated ICS process, while the same attacks reach and compromise the process 100% of the time when sent directly. This confirms the guard's value as a real-time data-integrity control for Modbus/TCP.

That protection, however, is conditional on all traffic actually passing through the guard. The lab's own architecture leaves a direct path (port 5020) open in parallel with the guarded path (port 5021), and nothing in the current design prevents a client from using the direct path. Combined with the complete absence of authentication and authorisation on every service, and the guard's blindness to read requests and to denial-of-service activity, the guard should be understood as one layer of defence-in-depth for write-integrity - not as a complete OT security solution. The recommendations in Section 4.4 (authentication, authorisation, retiring or restricting the direct path, rate limiting, correlated-value checks, and adaptive thresholds) would need to be implemented before this design could be considered for anything beyond an educational setting.

References

- NIST SP 800-82 Rev. 3 - Guide to Operational Technology (OT) Security. https://csrc.nist.gov/pubs/sp/800/82/r3/final
- Modbus Organization - Modbus Application Protocol Specification V1.1b3. https://modbus.org
- MITRE ATT&CK for ICS Matrix. https://attack.mitre.org
- ACICP302 - ICS Network Security and Attacks, Lab Repository (ICDFA). https://github.com/icdfa/ACICP302ICS-Network-Security-and-Attacks (commit 2a66f9bcf253eb3fe31c68ad52794cd535421da8)
- pymodbus Library Documentation (v3.6.9). https://pymodbus.readthedocs.io